Viva Preparation: Questions and Answers

eBPF bufferbloat detection + adaptive AQM prototype | CSC 303, IIT Roorkee

Cheat sheet: numbers and names to remember

Item	Value
Topology	h1 (10.0.1.2) -- router r -- h2 (10.0.2.2); three network namespaces joined by veth pairs
Bottleneck	r-eth1, htb shaper at 10 Mbit, leaf queue pfifo with 1000 packets (about 1.2 s of buffer)
Base RTT	20 ms (netem on h2-eth0, return direction only)
eBPF hooks	tracepoints net:net_dev_queue (enter) and net:net_dev_start_xmit (leave)
eBPF maps	enq_ts (packet -> time), flows (5-tuple -> bytes), slots (64 x 100 ms ring)
Detect rule	rate ≥ 85% of 10 Mbit AND min sojourn > 50 ms AND ping inflation > 50 ms, held 2 s
Timers	tick 0.5 s, persist 2 s, exit-persist 3 s, dwell 6 s, cooldown / idle 8 s
Hysteresis	enter 50 ms, exit 20 ms
Ladder	pfifo -> fq_codel -> cake (one rung at a time; back to pfifo when the link is idle 8 s)
Result p50 RTT	static pfifo 953.5 ms vs adaptive 21.1 ms (about 45x lower)
Result p95 / p99	1252 / 1262 ms vs 163 / 220 ms
Goodput	9.32 vs 9.47 Mbit/s (no throughput lost)
Drops / retrans	343 / 341 vs 88 / 281
Reaction time	load starts 7.7 s; bad from 8.2 s; DEGRADED 10.2 s; fq_codel at 10.8 s (about 3 s total)

How to answer: lead with the one-sentence answer, then give one detail from *your* project (a number, a file, a line). If you don't know something, say what you would do to find out. Never invent a result: "we have not measured that yet; it is planned" is a good answer.

A. The problem and the big picture

Q1. Explain your project in one minute.

Bufferbloat is when a network device holds a very large queue, so whenever the link is busy every packet waits a long time and latency jumps from milliseconds to seconds. We built a closed control loop on Linux. An **eBPF** program inside the kernel measures how long each packet waits in the bottleneck queue. A **Python controller** detects a standing queue and automatically **switches the queue discipline** from pfifo to fq_codel (and to cake if needed), then switches back when the load is gone. In our testbed it cut median latency under load from about 950 ms to about 21 ms without losing throughput.

Q2. What is bufferbloat?

Excessive buffering in a network device, which causes high latency under load. Loss-based TCP keeps increasing its sending rate until a packet is dropped. With a huge FIFO buffer, drops happen only when the buffer is full, so TCP keeps the buffer full all the time. Every other packet, such as a ping, a game or a VoIP call, must wait behind it.

If they push: In our testbed: 1000 packets x 1500 bytes x 8 / 10 Mbit = 1.2 s of queue, which is why ping reached about 1 s.

Q3. Why is a big buffer bad? Isn't more buffer better because it avoids loss?

A buffer is useful only for absorbing short bursts. Once it holds a *standing* queue that never drains, it adds delay without adding throughput: the link is already 100% busy. Our results show this. fq_codel kept goodput at 9.47 Mbit/s with a much smaller queue.

Q4. What is a qdisc?

A queueing discipline: the Linux kernel component that decides how packets wait and in what order they leave an interface. Every interface has one. It is configured with the `tc` command.

Q5. What is AQM?

Active Queue Management. Instead of dropping packets only when the queue is full ("tail drop", which is what pfifo does), the queue drops or ECN-marks packets *early*, based on delay. That signals TCP to slow down before a large queue builds. CoDel, PIE and RED are AQMs.

Q6. What is the novelty? fq_codel and cake already exist.

We do not invent a new AQM. Our contribution is the **measurement and control loop around them**: precise in-kernel delay measurement for *any* qdisc with eBPF, a detector for standing queues, and a controller that picks and verifies the policy at run time, with rollback if a change makes things worse.

If they push: Being honest: whether adapting beats always running fq_codel is exactly the experiment planned for the final project.

Q7. Linux already defaults to fq_codel on many distros. Why does bufferbloat still happen?

The qdisc that matters is the one at the **bottleneck**, which is usually the home router, modem or ISP equipment, often with a large FIFO. A fast laptop NIC running fq_codel does not help if the queue builds somewhere else. That is why our controller runs on the router (`bb-r`) at the bottleneck interface.

B. Queue disciplines: pfifo, CoDel, fq_codel, cake, htb, netem

Q8. What is pfifo?

Packet FIFO: a single first-in, first-out queue with a limit in packets (we use 1000). When it is full, new packets are dropped (tail drop). It has no delay control, so it is our bloated baseline.

Q9. How does CoDel work?

CoDel (Controlled Delay) looks at sojourn time, how long each packet waited. If the **minimum** sojourn stays above **target (5 ms)** for a whole **interval (100 ms)**, there is a standing queue. CoDel then starts dropping packets at dequeue, increasing the drop rate until the delay falls.

If they push: We borrowed this idea: our eBPF window is 100 ms and we detect on the minimum sojourn per window.

Q10. What does fq_codel add?

Flow queueing. Packets are hashed by 5-tuple into many separate queues (1024 by default), served round-robin (DRR), and CoDel runs on each queue. A light flow like ping no longer waits behind a bulk flow, and new or sparse flows get priority.

Q11. What is cake, and why is it the top rung?

CAKE (Common Applications Kept Enhanced) combines flow queueing, a CoDel variant (COBALT, which also handles unresponsive flows), per-host fairness, DiffServ priority and ACK filtering. It is the most capable but costs more CPU, so it is the last step on the ladder.

Q12. What are htb and netem doing in your testbed?

htb (Hierarchical Token Bucket) is a shaper. It limits r-eth1 to 10 Mbit, creating a real bottleneck, because a veth link has no speed limit of its own. Our managed queue is the htb leaf under class `1:1`. **netem** adds a fixed 20 ms delay to emulate a real path RTT.

Q13. Why is netem on h2 (the return path) and not on the bottleneck?

netem has its own internal queue. On the bottleneck direction it would mix its queue with the one we are measuring. On the return path it only delays ACKs and ping replies, giving a clean 20 ms base RTT.

Q14. Why did you turn off TSO, GSO and GRO?

Otherwise the kernel passes 64 KB "super-packets" across the virtual links, and a 1000-packet queue would hold about 64 MB instead of 1.5 MB. Disabling them makes every packet about 1500 bytes, like a real wire. See `testbed.sh` lines 43-49.

Q15. What happens to packets when you replace the qdisc?

Packets queued in the old qdisc are discarded (comment at `controller.py` line 58). That is a one-time loss, and TCP retransmits them. Because those packets were sitting in a bloated queue anyway, flushing them actually removes the delay immediately: the sojourn dropped from 181 ms to 0 ms in one tick.

C. eBPF

Q16. What is eBPF?

Extended Berkeley Packet Filter: a way to run small, sandboxed programs **inside the Linux kernel** without changing kernel code or loading a kernel module. Programs attach to hooks (tracepoints, kprobes, tc, XDP), run on events, and share data with user space through **maps**.

Q17. How is it safe to run your code in the kernel?

Before loading, the kernel **verifier** checks the program: every memory access must be safe, loops must be bounded, and only allowed helper functions can be called. Then it is JIT-compiled to native code. Our program only reads packet metadata and updates maps. It never changes or drops traffic.

Q18. What is a BPF map? Which ones do you use?

A key-value table in kernel memory that both the eBPF program and user space can read. We use three. `enq_ts`: packet address -> enqueue time (LRU hash). `flows`: 5-tuple -> bytes (LRU hash). `slots`: an array of 64 slots of 100 ms each with count, sum, min and max sojourn (a ring buffer).

Q19. Which hooks do you attach to, and why these?

`net:net_dev_queue` fires when a packet is handed to the qdisc, and `net:net_dev_start_xmit` fires when it leaves the qdisc for the driver. The difference is the exact time the packet spent in the queue, and it works for **any** qdisc, including pfifo, which keeps no delay statistics itself.

Q20. Why tracepoints instead of a tc-BPF program or kprobes?

A tc-BPF program runs at one point and cannot see when the packet *leaves* the qdisc, so it cannot measure sojourn. kprobes attach to internal kernel functions that can change between versions. Tracepoints are a **stable** interface placed exactly at enqueue and dequeue.

Q21. What is BCC?

BPF Compiler Collection, a toolkit that compiles our C file with clang at run time, loads it, attaches the probes and gives Python access to the maps. In `controller.py` this is the `BPF(src_file="observer.bpf.c", ...)` call (line 76).

If they push: Its downside is that it needs clang and kernel headers on the target. The final version could use libbpf with CO-RE (compile once, run everywhere).

Q22. How do you know which interface a packet belongs to?

Tracepoints fire for every interface in every namespace. `on_target()` reads the packet's device name and compares it with "r-eth1" as two 64-bit numbers with a mask. The mask includes the terminating zero, so r-eth10 would not match.

Q23. Why aggregate in the kernel instead of sending every packet to user space?

Overhead. At 10 Mbit that is about 830 packets per second, and on real links millions. The kernel keeps only count, sum, min and max per 100 ms window, and Python reads a few small slots every 0.5 s.

Q24. What if a packet is dropped inside the queue and never leaves?

Its timestamp stays in `enq_ts`, but that map is an **LRU** hash, so old entries are evicted automatically and the table cannot fill up.

Q25. What is the overhead of your eBPF program?

Not measured yet, and that is listed as a limitation. Per packet it does two map operations and a few memory reads, so we expect it to be very small, but we will measure it with `bpftool prog profile` or `perf` in the final project.

Q26. Do you handle IPv6?

Sojourn is measured for **all** packets on r-eth1, including IPv6. Only the per-flow counter `count_flow()` is IPv4-only, and that is used for display, not decisions.

D. The controller and its design choices

Q27. Walk me through one control-loop tick.

Every 0.5 s: read tc statistics (link rate, backlog, drops); read the eBPF slots (min, avg and max sojourn) and the flows map; get the median ping RTT and compute inflation over the baseline; call `Controller.step()`, which classifies the tick and may switch the qdisc; write a CSV row and print the live table.

Q28. What are the controller's states?

NORMAL (all fine); DEGRADED (standing queue confirmed); MITIGATING (just switched, waiting 6 s and then judging the result); RECOVERED (the fix works and is kept). From RECOVERED: degraded again goes to DEGRADED, and 8 s of idle link goes back to pfifo and NORMAL.

```
NORMAL --bad 2 s--> DEGRADED --next rung--> MITIGATING --dwell 6 s, good 3 s--> RECOVERED
  ^                                           | still bad: escalate | worse 25%: roll back |
  +------------------------ link idle 8 s: back to pfifo <-----------------------------------+
```

Q29. Why do you need three conditions to detect bloat?

To avoid false alarms. **Saturation**: a standing queue can only form on a full link. **Min sojourn > 50 ms**: the queue really is standing, not a burst. **Ping inflation > 50 ms**: users actually feel the delay. All three must hold for 2 s.

Q30. Why use the MINIMUM sojourn and not the average?

A short burst raises the average and maximum for a moment and then drains. That is the buffer doing its job. If even the **minimum** in every 100 ms window is above 50 ms, then every packet waited, so the queue never drained. That is CoDel's definition of a standing queue.

Q31. What is hysteresis and why do you use it?

Using two thresholds: enter at 50 ms and exit at 20 ms. With a single threshold, a delay hovering around it would flip the state back and forth (flapping). The gap makes decisions stable.

Q32. What is the dwell time for?

Right after a switch, the measurements reflect the transient of the change itself. The controller waits 6 s before judging whether the new qdisc helped.

Q33. Why revert to pfifo only when the link is idle, and not when delay is low?

Low delay is the **result** of fq_codel. If we reverted because delay was low, the bloat would return, we would detect it and switch again, and the controller would flap forever. So we revert only when the **cause**, the load, is gone (`controller.py` lines 246-250).

Q34. What does rollback do?

If after the dwell the delay is more than 25% **worse** than before the switch, the controller goes back to the previous qdisc and locks out further changes for 8 s, so a bad decision is undone automatically.

If they push: It has not been triggered in our runs, because fq_codel always helped here.

Q35. Why move only one rung at a time?

The smallest change that fixes the problem is preferred (fq_codel is cheaper than cake), and one change at a time means each effect can be verified and rolled back.

Q36. Why does the controller need ping if eBPF measures the delay?

eBPF shows the queue is long. Ping inflation confirms it is **harming user traffic**. That second signal reduces false positives. The controller still works without ping (`--no-probe`), using eBPF and saturation alone.

Q37. How is the RTT baseline and inflation computed?

Baseline = the minimum ping RTT over the last 120 s (the empty-queue RTT, about 20 ms). Inflation = the current median RTT minus the baseline, which is the extra delay caused by queueing.

Q38. What happens if eBPF fails to load?

The controller falls back to an estimate: delay = backlog bytes x 8 / link rate (Little's law, the "derived" column), and keeps working (`controller.py` lines 288-293).

Q39. Why Python for the controller, and why 0.5 s?

The per-packet hot path is in the kernel (eBPF). The controller makes one decision every 0.5 s, so Python is fast enough and easy to change. 0.5 s is five 100 ms windows: fast enough to react within seconds and slow enough to average out noise.

Q40. How did you choose the thresholds (50 ms, 20 ms, 85%, 2 s)?

They are reasonable illustrative values: 50 ms is clearly noticeable for interactive traffic, and 20 ms is close to the base RTT. They are **not tuned**. A sensitivity analysis is part of the final plan.

E. Results

Q41. What are your results?

Same traffic, 30 s of bulk TCP. Static pfifo: ping p50 953.5 ms, p95 1252 ms. Adaptive: p50 21.1 ms, p95 163 ms. Goodput 9.32 vs 9.47 Mbit/s, so no throughput lost. Drops fell from 343 to 88, and the controller made 2 switches (to fq_codel and back to pfifo).

Q42. Why are the adaptive p95 and p99 still high (163 and 220 ms)?

They come from the roughly 3 s between the load starting and the switch. The controller deliberately waits for 2 s of evidence before acting. After the switch, RTT sits at about 21 ms.

If they push: Lowering --persist would react faster but risk false alarms. That is the trade-off to study.

Q43. How fast does it react?

In the latest run the load started at 7.7 s, all three bad conditions held from 8.2 s, DEGRADED was declared at 10.2 s (2 s persistence), and fq_codel was installed on the next tick at 10.8 s. That is about 3 s from load to fix.

Q44. Why did throughput not drop when you used fq_codel?

The link stays fully used. TCP only needs a small queue to keep it busy. fq_codel removes the *excess* queue, not the useful part.

Q45. Why does the bulk flow's own delay matter? Ping is low under fq_codel anyway.

Good point. Ping is a sparse flow, and fq_codel gives sparse flows priority, so ping alone understates the harm. That is why we also plot the eBPF sojourn of the bulk flow, which stays around 5 ms under fq_codel, CoDel's target.

Q46. Are your results statistically significant?

Not yet. They are single runs per condition. The difference (950 ms vs 21 ms) is very large, but the final project will repeat each run 10 or more times and report means with confidence intervals.

F. Testbed and demo

Q47. What is a network namespace?

An isolated copy of the Linux network stack with its own interfaces, IPs, routes and qdiscs. We create three: bb-h1, bb-r and bb-h2. They behave like two hosts and a router, but all run in one kernel. Docker containers use the same feature.

Q48. What are h1, h2 and r?

h1 (10.0.1.2) is the sender: iperf3 client and ping. r is the router with the 10 Mbit bottleneck on r-eth1. h2 (10.0.2.2) is the receiver: iperf3 server, and it answers pings.

Q49. What is a veth pair?

A virtual Ethernet cable: two interfaces where whatever is sent into one comes out of the other. We use one between h1 and r and one between r and h2.

Q50. Why not use a simulator like ns-3 or Mininet?

Namespaces run the **real** Linux TCP stack, real qdiscs and real kernel tracepoints. Our eBPF program hooks into exactly the code a real Linux router would run. A simulator would not run eBPF at all. (Mininet itself is built on namespaces.)

Q51. Why does it run in a VM on your Mac?

Namespaces, qdiscs and eBPF are Linux kernel features. macOS does not have them. We use a Lima VM running Debian 12 with kernel 6.1, and the project folder is shared with the Mac.

Q52. What do the columns in the live output mean?

t time; **state** controller state; **qdisc** current queue; **rate** link throughput; **backlog** bytes queued; **derived** backlog/rate estimate; **sojourn** eBPF-measured queue wait (main signal); **rtt** ping RTT; **infl** RTT minus baseline; **drops** queue drops; **top flow** heaviest flow from eBPF.

Q53. What is the difference between the static and adaptive runs?

Same traffic. In the static run the controller is started with `--observe-only`: it measures and detects but never changes the qdisc. The adaptive run lets it act. This gives a fair before/after comparison.

Q54. What tools generate the traffic?

iperf3 (one bulk TCP flow from h1 to h2 for 30 s) and ping (5 per second from h1 to h2) as the latency-sensitive "victim" flow.

G. Limitations and future work

Q55. What are the limitations of your prototype?

Single runs; no comparison against static fq_codel or cake; cake, escalation and rollback were never exercised; only qdisc switching, no parameter tuning; illustrative thresholds; eBPF overhead not measured; only one TCP flow on a fixed 10 Mbit link; virtual testbed only.

Q56. Why is it a prototype and not the full project?

We built the riskiest part first, to prove that eBPF can measure queue delay accurately and that switching qdiscs live works end to end. It does. What remains is mainly evaluation, more scenarios and tuning.

Q57. How will you proceed to the final project?

(1) Rigorous evaluation: repeated runs, compared against static pfifo, fq_codel and cake. (2) Harder scenarios: multiple flows, a UDP flood (which should push it to cake), changing link capacity, different RTTs, CUBIC vs BBR. (3) Smarter control: tune fq_codel target and interval, and use the per-flow eBPF data. (4) Measure eBPF overhead, consider libbpf/CO-RE, and test on real hardware.

Q58. If fq_codel always fixes it, why not just always run fq_codel?

That is the key question, and our most important next experiment. Possible advantages of adapting are: using cake's stronger features only when needed (for example, against unresponsive UDP floods), since it costs more CPU; tuning parameters to the measured conditions; and observability, because the eBPF detector tells the operator *when* and *why* the network is bloated. If static fq_codel turns out to be just as good, that is a valid result too.

Q59. Could this run on a real router?

Yes in principle. OpenWrt and modern Linux routers support eBPF and all these qdiscs. The controller needs only `tc` and read access to the maps. It would need the libbpf/CO-RE version and an overhead measurement first.

Q60. What about ECN, L4S or BBR?

fq_codel and cake support ECN marking instead of dropping. We have not enabled or tested it yet. BBR is a delay-based TCP that builds smaller queues, so testing CUBIC vs BBR is in the plan. L4S is a newer low-latency architecture that could be future work.

H. Code questions they may point at

Q61. Where is the eBPF program, and where is it loaded?

`observer.bpf.c`. It is loaded by `controller.py`, class `Observer`, line 76: `BPF(src_file=...)`. There is no separate command; running the controller loads it.

Q62. Where is the qdisc actually changed?

`controller.py` lines 56-62, `Qdisc.apply()`: `tc qdisc replace dev r-eth1 parent 1:1 handle N: fq_codel`. A new handle is used each time because tc cannot change a qdisc's type in place.

Q63. Where is the detection rule?

`controller.py` lines 200-204 in `Controller.step()`: the definitions of `saturated`, `bad`, `good` and `idle`.

Q64. How is sojourn computed in the kernel?

`observer.bpf.c` lines 100-134. Look up the packet's enqueue timestamp, compute now minus that timestamp, delete the entry, and fold the value into the 100 ms slot `epoch % 64`.

Q65. Why is there an atomic add in count_flow but not in the slot update?

Enqueue can happen on several CPUs at once, so the per-flow byte counter needs `__sync_fetch_and_add`. Dequeue from one device queue is serialised by the kernel, so the slot update cannot race.

Q66. How do Python and the kernel agree on the 100 ms windows?

`bpf_ktime_get_ns()` and Python's `time.monotonic_ns()` use the same monotonic clock, and both sides use the same `WINDOW_NS`, which Python passes to the C code as a `-D` flag.

Last tips

- Keep the demo running while answering, and point at the live table when you can.
- If you are asked something you did not do, say: "Not in the prototype, and here is how I would do it."
- Open the code at the line numbers above when the examiner asks "where is that?"
- Be ready to draw the topology: h1 -- r -- h2, with the bottleneck queue on r-eth1.